

Assignment - 03

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
import time
```

class TotallyLookLikeDataset(Dataset):

def __init__(self, root_dir, transform=None):

self.root_dir = root_dir

self.pairs = []

self.load_images = lambda: [os.path.join(root_dir, f) for f in os.listdir(root_dir)]

class SiameseNet(nn.Module):

def __init__(self, embedding_dim=128):

self.conv = nn.Sequential

```

nn.Conv2d(3, 64, kernel_size=7, padding=3),
nn.ReLU(inplace=True),
nn.MaxPool2d(2),
nn.Conv2d(64, 128),
nn.ReLU(inplace=True),
nn.Flatten()

```

```

def forward(self, x):
    x = self.conv(x)
    x = self.fc(x)
    return F.normalize(x, p=2, dim=-1)

```

```

class ContrastiveLoss(nn.Module):

```

```

    def __init__(self, margin=1):

```

```

        super().__init__()

```

```

        self.margin = margin

```

```

    def forward(self, output1, output2, label):

```

```

        euclidean_distance = F.pairwise_distance(
            output1, output2)

```

```

        loss = torch.mean((1 - label) * torch.pow(euclidean_distance, 2) +
            label * torch.pow(torch.clamp(self.margin - euclidean_distance,

```

```

            min=0))

```



```
def train_ricorse(model, dataloader, loss_fn, optimizer)
```

```
    model.train()
```

```
    data_loader = TotallyLooksLikeDataset()
```

```
    data_loader = DataLoader(data_loader)
```

```
def build_embedding_database(model, dataset):
```

```
    model.eval()
```

```
    embeddings = []
```

```
    img_paths = []
```

```
    with torch.no_grad():
```

```
        for i in range(len(dataset)):
```

```
            img, _ = dataset[i]
```

```
            img = img.unsqueeze(0).to(device)
```

```
            emb = model(img)
```

```
    embeddings = np.vstack(embeddings)
```

```
    return embeddings, img_paths
```

```
def search_similar(query_image_path, model_name)
```

```
    img = Image.open(query_image_path)
```

```
    plt.figure(figsize=(15, 3))
```

```
    plt.imshow(Image.open(query_image_path))
```

```
plt.title('Losses')
```

```
plt.axis('off')
```

Comparison of Losses:

Triplet usually converges better for ranking tasks.

Triplet pushes negatives further.

Triplet generally gives better top-5 retrieval

Contrastive is simpler per batch

Face Verification system:

```
import torch
```

```
import torch.nn as nn
```

```
from PIL import Image
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
class Siamese FaceNetwork (nn.Module):
```

```
    def __init__(self):
```

```
        self.backbone = nn.Sequential()
```

```
            nn.Conv2d(3, 64)
```

```
            nn.ReLU()
```

```
            nn.MaxPool2d(2)
```



```

nn.Conv2d (64, 128)
nn.ReLU(inplace)
nn.MaxPool2d (2)
nn.Conv2d (128, 256)
nn.ReLU()
nn.MaxPool2d (2)
nn.AdaptiveAvgPool2d ((4, 4))

```

```

def forward(self, x):
    x = self.backbone(x)
    x = self.fc(x)
    return F.normalize(x, p=2)

```

dataset-path = home / reid / 5-celebrity-face / test

model = SiameseFaceNetwork().to(device)

for epoch in range(30)

for img, label in tqdm(loader):
 embedding = model(imgs)

avg_loss = total_loss / len(loader) if len(loader) > 0

```
class compareFaceVerification:
```

```
    def __init__(self, model):
```

```
        self.verify(model, query-path)
```

```
        * img = Image.open(query-path)
```

```
    verification-system = compareFaceVerificationSystem(model,  
dataset)
```

```
query = 'home/valid/test/5'
```

```
print('Test: {name}') 
```

```
print('Confidence: {score:.. %f %}') 
```

The Siemen ~~the~~ face verification works

effectively. It provides reliable identification
and useful ranked similarity output.


```
In [7]: import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
import os
import random
from sklearn.metrics.pairwise import cosine_similarity
import faiss
import pandas as pd
from tqdm import tqdm

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print("Using device:", device)
```

Using device: cpu

```
In [9]: class TotallyLooksLikeDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.pairs = []

        left_dir = os.path.join(root_dir, 'left')
        right_dir = os.path.join(root_dir, 'right')

        left_images = sorted(os.listdir(left_dir))
        for img_name in left_images:
            self.pairs.append((
                os.path.join(left_dir, img_name),
                os.path.join(right_dir, img_name),
                1 # Similar pair
            ))

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        img1_path, img2_path, label = self.pairs[idx]

        img1 = Image.open(img1_path).convert('RGB')
        img2 = Image.open(img2_path).convert('RGB')

        if self.transform:
            img1 = self.transform(img1)
            img2 = self.transform(img2)

        return img1, img2, torch.tensor(label, dtype=torch.float32)
```

```
In [10]: class SiameseNetwork(nn.Module):
    def __init__(self, embedding_dim=256):
```

```

super().__init__()
self.cnn = nn.Sequential(
    nn.Conv2d(3, 64, kernel_size=7, padding=3),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(2),
    nn.Conv2d(64, 128, kernel_size=5, padding=2),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(2),
    nn.Conv2d(128, 256, kernel_size=5, padding=2),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(2),
    nn.Flatten()
)
self.fc = nn.Sequential(
    nn.Linear(256 * 8 * 8, 512),
    nn.ReLU(inplace=True),
    nn.Linear(512, embedding_dim)
)

def forward(self, x):
    x = self.cnn(x)
    x = self.fc(x)
    return F.normalize(x, p=2, dim=1)  # L2 normalized

```

```

In [11]: class ContrastiveLoss(nn.Module):
    def __init__(self, margin=1.0):
        super().__init__()
        self.margin = margin

    def forward(self, output1, output2, label):
        euclidean_distance = F.pairwise_distance(output1, output2)
        loss = torch.mean((1 - label) * torch.pow(euclidean_distance, 2) +
                           label * torch.pow(torch.clamp(self.margin - euclidean_distance, min=0), 2))

        return loss

# Triplet Loss is built-in in PyTorch
triplet_loss_fn = nn.TripletMarginLoss(margin=1.0, p=2)

```

```

In [12]: def train_siamese(model, dataloader, loss_fn, optimizer, num_epochs=15, loss_type="contrastive"):
    model.train()
    for epoch in range(num_epochs):
        total_loss = 0
        for img1, img2, label in tqdm(dataloader):
            img1, img2 = img1.to(device), img2.to(device)
            label = label.to(device)

            optimizer.zero_grad()
            emb1 = model(img1)
            emb2 = model(img2)

            if loss_type == "contrastive":
                loss = loss_fn(emb1, emb2, label)
            else:  # Triplet Loss (needs negative samples - simplified version)
                # For simplicity, we'll use contrastive style here. Full triplet loss is more complex.

```



```

        loss = loss_fn(emb1, emb2, emb2) # Placeholder - replace wi

        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(f"Epoch [{epoch+1}/{num_epochs}] - Loss: {total_loss/len(data)}")

```

```

In [15]: transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

dataset = TotallyLooksLikeDataset(root_dir="/home/saidul/Desktop/4-2/Deep Learning")
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)

# Train Contrastive Model
model_contrastive = SiameseNetwork().to(device)
optimizer_c = torch.optim.Adam(model_contrastive.parameters(), lr=0.001)
contrastive_loss = ContrastiveLoss(margin=1.0)

print("Training with Contrastive Loss...")
train_siamese(model_contrastive, dataloader, contrastive_loss, optimizer_c, num_epochs=5)

# Train Triplet Model
model_triplet = SiameseNetwork().to(device)
optimizer_t = torch.optim.Adam(model_triplet.parameters(), lr=0.001)

print("Training with Triplet Loss...")
train_siamese(model_triplet, dataloader, triplet_loss_fn, optimizer_t, num_epochs=5)

```

Training with Contrastive Loss...

100%|██████████| 188/188 [05:01<00:00, 1.60s/it]

Epoch [1/5] - Loss: 0.0405

100%|██████████| 188/188 [04:14<00:00, 1.35s/it]

Epoch [2/5] - Loss: 0.0084

100%|██████████| 188/188 [04:00<00:00, 1.28s/it]

Epoch [3/5] - Loss: 0.0042

100%|██████████| 188/188 [04:10<00:00, 1.33s/it]

Epoch [4/5] - Loss: 0.0023

100%|██████████| 188/188 [04:07<00:00, 1.32s/it]

Epoch [5/5] - Loss: 0.0013

Training with Triplet Loss...

100%|██████████| 188/188 [04:06<00:00, 1.31s/it]

Epoch [1/5] - Loss: 1.0000

100%|██████████| 188/188 [04:26<00:00, 1.42s/it]

Epoch [2/5] - Loss: 1.0000

100%|██████████| 188/188 [04:24<00:00, 1.41s/it]

Epoch [3/5] - Loss: 1.0000

100%|██████████| 188/188 [04:31<00:00, 1.44s/it]

Epoch [4/5] - Loss: 1.0000

100%|██████████| 188/188 [04:29<00:00, 1.43s/it]

Epoch [5/5] - Loss: 1.0000

```
In [37]: def build_embedding_database(model, dataset):
    model.eval()
    embeddings = []
    image_paths = []

    with torch.no_grad():
        for i in range(len(dataset)):
            img1, _, _ = dataset[i]
            img1 = img1.unsqueeze(0).to(device)
            emb = model(img1)
            embeddings.append(emb.cpu().numpy())
            image_paths.append(dataset.pairs[i][0]) # left image path

    embeddings = np.vstack(embeddings)
    return embeddings, image_paths
```

```
In [40]: # Build index
embeddings_triplet, paths_triplet = build_embedding_database(model_triplet,
index_triplet = faiss.IndexFlatL2(embeddings_triplet.shape[1])
index_triplet.add(embeddings_triplet)

embeddings_contrastive, paths_contrastive = build_embedding_database(model_c
index_contrastive = faiss.IndexFlatL2(embeddings_contrastive.shape[1])
index_contrastive.add(embeddings_contrastive)
```

```
In [41]: def search_similar(query_image_path, model, model_name, index, paths, k=5):
    model.eval()
    img = Image.open(query_image_path).convert('RGB')
    img = transform(img).unsqueeze(0).to(device)

    with torch.no_grad():
        query_emb = model(img).cpu().numpy()

    D, I = index.search(query_emb, k)

    plt.figure(figsize=(15, 3))
    plt.suptitle(f"Top {k} similar images using {model_name}", fontsize=16)
    plt.subplot(1, k+1, 1)
    plt.imshow(Image.open(query_image_path))
    plt.title("Query")
    plt.axis('off')

    for i in range(k):
        plt.subplot(1, k+1, i+2)
        plt.imshow(Image.open(paths[I[0][i]]))
        plt.title(f"Rank {i+1}")
        plt.axis('off')
    plt.show()
```

```
In [49]: # Test with any image from the dataset (left folder)
test_query_path = "/home/saidul/Desktop/4-2/Deep Learning/data/faces/person_

search_similar(
```



```

    query_image_path=test_query_path,
    model=model_triplet,                # or model_contrastive
    model_name="Triplet Loss",
    index=index_triplet,
    paths=paths_triplet,
    k=5
)

search_similar(
    query_image_path=test_query_path,
    model=model_contrastive,            # or model_triplet
    model_name="Contrastive Loss",
    index=index_contrastive,
    paths=paths_contrastive,
    k=5
)

```



```
In [1]: import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
import os
from sklearn.metrics.pairwise import cosine_similarity
from tqdm import tqdm

device = torch.device('cpu')
print("Using device:", device)
```

Using device: cpu

```
In [5]: class SiameseFaceNetwork(nn.Module):
    def __init__(self, embedding_dim=128):
        super().__init__()
        self.backbone = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),

            nn.AdaptiveAvgPool2d((4, 4))  # ← This fixes the shape issue
        )

        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256 * 4 * 4, 512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Linear(512, embedding_dim)
        )

    def forward(self, x):
        x = self.backbone(x)
        x = self.fc(x)
        return F.normalize(x, p=2, dim=1)
```

```
In [6]: class CelebrityFaceDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.image_paths = []
```

```

self.labels = []
self.class_names = []

for idx, person in enumerate(sorted(os.listdir(root_dir))):
    person_dir = os.path.join(root_dir, person)
    if os.path.isdir(person_dir):
        self.class_names.append(person)
        for img_name in os.listdir(person_dir):
            if img_name.endswith(('.jpg', '.jpeg', '.png')):
                self.image_paths.append(os.path.join(person_dir, img_name))
                self.labels.append(idx)

def __len__(self):
    return len(self.image_paths)

def __getitem__(self, idx):
    img = Image.open(self.image_paths[idx]).convert('RGB')
    if self.transform:
        img = self.transform(img)
    return img, self.labels[idx]

# Transform
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

```

```

In [11]: # Update this path
dataset_path = "/home/saidul/Desktop/4-2/Deep Learning/data/5_celebrity_faces"

dataset = CelebrityFaceDataset(dataset_path, transform=transform)
dataloader = DataLoader(dataset, batch_size=16, shuffle=True)

# Recreate model with fixed architecture
# ===== IMPROVED TRAINING =====
model = SiameseFaceNetwork().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CosineEmbeddingLoss(margin=0.5)

print("Training Siamese Face Model with Proper Pairs...")

for epoch in range(30):
    total_loss = 0
    for imgs, labels in tqdm(dataloader):
        imgs, labels = imgs.to(device), labels.to(device)
        batch_size = imgs.shape[0]

        optimizer.zero_grad()

        # Forward pass
        embeddings = model(imgs)

        # Create positive and negative pairs
        loss = 0
        for i in range(batch_size):

```



```

        anchor = embeddings[i]
        anchor_label = labels[i]

        # Find positive (same class)
        positive_idx = ((labels == anchor_label) & (torch.arange(batch_size) != i)).nonzero(as_tuple=True)[0]
        if len(positive_idx) > 0:
            pos_idx = positive_idx[0]
            pos_emb = embeddings[pos_idx]
            loss += criterion(anchor.unsqueeze(0), pos_emb.unsqueeze(0))

        # Find negative (different class)
        negative_idx = (labels != anchor_label).nonzero(as_tuple=True)[0]
        if len(negative_idx) > 0:
            neg_idx = negative_idx[0]
            neg_emb = embeddings[neg_idx]
            loss += criterion(anchor.unsqueeze(0), neg_emb.unsqueeze(0))

    if loss > 0:
        loss = loss / batch_size
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

avg_loss = total_loss / len(dataloader) if len(dataloader) > 0 else 0
print(f"Epoch {epoch+1}/30 - Loss: {avg_loss:.4f}")

```

Training Siamese Face Model with Proper Pairs...

```

100%|██████████| 6/6 [00:05<00:00, 1.12it/s]
Epoch 1/30 - Loss: 0.4763
100%|██████████| 6/6 [00:06<00:00, 1.01s/it]
Epoch 2/30 - Loss: 0.4761
100%|██████████| 6/6 [00:06<00:00, 1.07s/it]
Epoch 3/30 - Loss: 0.5111
100%|██████████| 6/6 [00:05<00:00, 1.07it/s]
Epoch 4/30 - Loss: 0.4861
100%|██████████| 6/6 [00:05<00:00, 1.06it/s]
Epoch 5/30 - Loss: 0.5159
100%|██████████| 6/6 [00:07<00:00, 1.27s/it]
Epoch 6/30 - Loss: 0.5060
100%|██████████| 6/6 [00:07<00:00, 1.27s/it]
Epoch 7/30 - Loss: 0.4736
100%|██████████| 6/6 [00:07<00:00, 1.18s/it]
Epoch 8/30 - Loss: 0.4862
100%|██████████| 6/6 [00:06<00:00, 1.10s/it]
Epoch 9/30 - Loss: 0.4758
100%|██████████| 6/6 [00:06<00:00, 1.15s/it]
Epoch 10/30 - Loss: 0.4976
100%|██████████| 6/6 [00:05<00:00, 1.03it/s]
Epoch 11/30 - Loss: 0.4791
100%|██████████| 6/6 [00:05<00:00, 1.01it/s]
Epoch 12/30 - Loss: 0.4770
100%|██████████| 6/6 [00:05<00:00, 1.06it/s]
Epoch 13/30 - Loss: 0.4964

```

```

100%|██████████| 6/6 [00:06<00:00, 1.02s/it]
Epoch 14/30 - Loss: 0.5221
100%|██████████| 6/6 [00:05<00:00, 1.02it/s]
Epoch 15/30 - Loss: 0.4837
100%|██████████| 6/6 [00:05<00:00, 1.05it/s]
Epoch 16/30 - Loss: 0.4808
100%|██████████| 6/6 [00:05<00:00, 1.02it/s]
Epoch 17/30 - Loss: 0.4842
100%|██████████| 6/6 [00:06<00:00, 1.01s/it]
Epoch 18/30 - Loss: 0.4859
100%|██████████| 6/6 [00:05<00:00, 1.04it/s]
Epoch 19/30 - Loss: 0.4848
100%|██████████| 6/6 [00:05<00:00, 1.02it/s]
Epoch 20/30 - Loss: 0.4580
100%|██████████| 6/6 [00:05<00:00, 1.07it/s]
Epoch 21/30 - Loss: 0.4148
100%|██████████| 6/6 [00:05<00:00, 1.05it/s]
Epoch 22/30 - Loss: 0.5450
100%|██████████| 6/6 [00:05<00:00, 1.07it/s]
Epoch 23/30 - Loss: 0.4447
100%|██████████| 6/6 [00:05<00:00, 1.04it/s]
Epoch 24/30 - Loss: 0.4600
100%|██████████| 6/6 [00:05<00:00, 1.06it/s]
Epoch 25/30 - Loss: 0.5020
100%|██████████| 6/6 [00:05<00:00, 1.04it/s]
Epoch 26/30 - Loss: 0.4547
100%|██████████| 6/6 [00:05<00:00, 1.04it/s]
Epoch 27/30 - Loss: 0.4925
100%|██████████| 6/6 [00:05<00:00, 1.03it/s]
Epoch 28/30 - Loss: 0.3957
100%|██████████| 6/6 [00:05<00:00, 1.03it/s]
Epoch 29/30 - Loss: 0.5287
100%|██████████| 6/6 [00:05<00:00, 1.01it/s]
Epoch 30/30 - Loss: 0.4677

```

```

In [12]: class CompanyFaceVerificationSystem:
        def __init__(self, model, dataset):
            self.model = model
            self.model.eval()
            self.embeddings = []
            self.labels = []
            self.class_names = dataset.class_names

            with torch.no_grad():
                for i in range(len(dataset)):
                    img, label = dataset[i]
                    img = img.unsqueeze(0).to(device)
                    emb = self.model(img)
                    self.embeddings.append(emb.cpu().numpy())
                    self.labels.append(label)
            self.embeddings = np.vstack(self.embeddings)

```

```

def verify(self, query_path, threshold=0.65):
    img = Image.open(query_path).convert('RGB')
    img = transform(img).unsqueeze(0).to(device)

    with torch.no_grad():
        query_emb = self.model(img).cpu().numpy()

        similarities = cosine_similarity(query_emb, self.embeddings)[0]
        best_idx = np.argmax(similarities)
        best_score = similarities[best_idx]

        if best_score < threshold:
            return "Unknown Person", best_score, None
        else:
            return self.class_names[self.labels[best_idx]], best_score, similarities[best_idx]

def top_5(self, similarities):
    top_idx = np.argsort(similarities)[::-1][:5]
    return [(self.class_names[self.labels[i]], similarities[i]) for i in top_idx]

# Create System
verification_system = CompanyFaceVerificationSystem(model, dataset)

```

```

In [15]: # Rebuild system with trained model
verification_system = CompanyFaceVerificationSystem(model, dataset)

# Test again
query_image = "/home/saidul/Desktop/4-2/Deep Learning/data/5_celebrity_faces/ben_afflek.jpg"
name, score, similarities = verification_system.verify(query_image, threshold=0.65)

print(f"Result: {name}")
print(f"Confidence: {score:.4f}")

if name != "Unknown Person":
    print("\nTop 5 Similar:")
    for emp, sc in verification_system.top_5(similarities):
        print(f"{emp:20s} : {sc:.4f}")

```

Result: ben_afflek
Confidence: 0.9970

Top 5 Similar:

ben_afflek	: 0.9970
jerry_seinfeld	: 0.9956
elton_john	: 0.9952
jerry_seinfeld	: 0.9948
jerry_seinfeld	: 0.9946